

Inigo Quilez :: articles :: per vertex ambient occlusion - 2005



As you know Ambient Occlusion is a widely used technique to improve the computer graphics lighting of the old shading models without resorting to full global illumination techniques. It just takes a few bytes of code (less than 1 kilobyte, as shown in some of the demos in this website) and the results are quite convincing. In fact, many productions for the film industry use ambient occlusion for simulating global illumination.

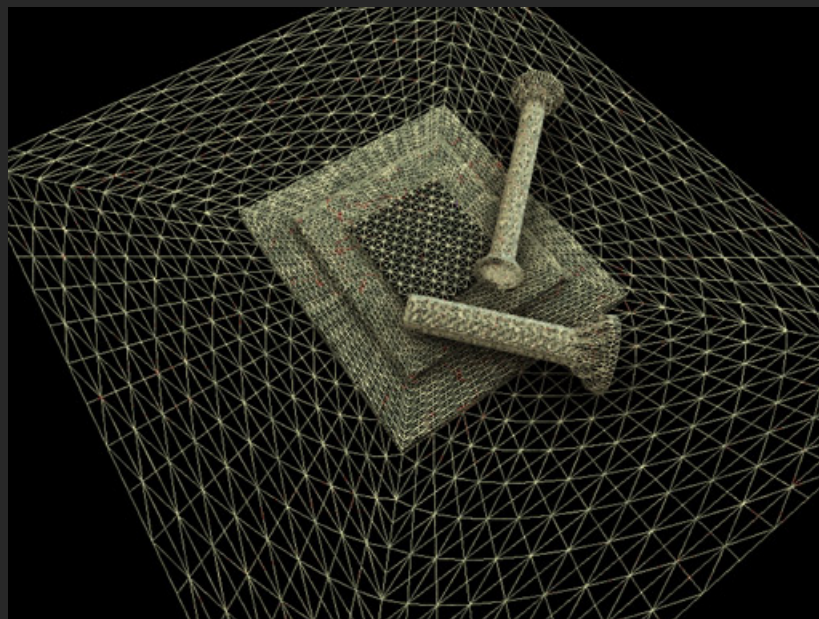
Just in case, I will explain quickly what ambient occlusion is. You will find hundred of references by just googling anyway (remember, google is our friend). So, Ambient Occlusion is the result of simplifying the rendering equation that describes the light interaction. But don't worry, you'll not find any integral in this fast-tutorial. However, you must know that in reality when a point on a surface of an object is lit, the light arriving to it normally comes from many directions. However in normal direct lighting models (what you do in OpenGL) only light coming from a few point or directional lights (normally no more than 8) is used. That's a very bad approximation to reality, where light can bounce several times around the scene before arriving to the point your pixel is representing. Actually, light is coming from "everywhere" to that point, and that "everywhere" means that we should "gather" the light arriving from everywhere to the point rather than sending light only from those 8 lights.

What ambient occlusion does is exactly that gathering of light from the everywhere, but in a very simplified mode. Simplifications are: light is not bouncing (even if this can be trivially simulated with AO) and thus all the geometry in the scene is blocking the light arriving from "everywhere" to that point (normally that "everywhere" means the sky). Thus, geometry blocks light arriving to the point. In case of having only one point light source, we also know this phenomena as "shadow". However, in the case of ambient occlusion we don't have the concept of shadow anymore, but of occlusion instead. If a point is completely surrounded by dense geometry then the point is probably quite occluded and no light arrives to it so it will be dark. If the point is however just facing the sky, with no geometry above it, the point will be white because all the light arrives to it. In between, of course, you have the complete spectrum of gray shades.

Thus, to get our ambient occlusion into the scene, we have three options. We calculate it for every pixel in the screen, we calculate it for many points in the surface and back it into textures, or we calculate it per vertex and then interpolate. First option is just too slow, and second one needs a uv-unwrapping technique, lot of processing time and lot of memory usage. So in some of the demos here I used per vertex calculation, what needs moderately tessellated geometry to catch all the details in the shading variations. Hopefully, the light

irradiance into a point is smoothly changing in space, so we will assume we can fake lighting with a low frequency ambient occlusion. This means that we don't really need so many polygons. For the static models of 195/95/256 we never used more than 60 thousand polygons per model.

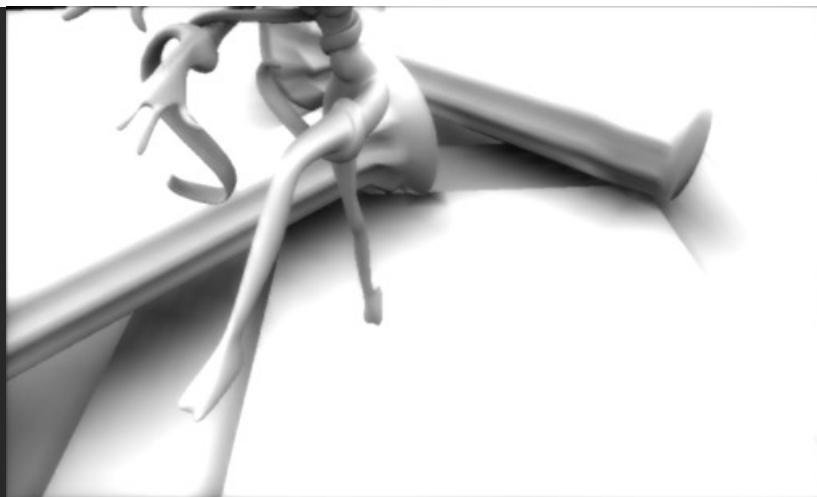
Something that can obviously be made is to use more vertices in those places where you expect higher frequencies, as shown in the next image below:



Adaptive tessellation level for those parts that more need it.

And remember that you don't need to specify lights on your scene, or at least it's not needed for basic results.

So, let's have a look to the results of all this. The image below shows how this Ambient Occlusion actually looks for one of the scenes of this project. You may find this rendering similar to the Brasil renderer, but this is just a shoot from the realtime 64 kilobytes demo. As opposed to most of the implementations of Ambient Occlusion, there is no raytracing at all for the calculations. Instead we use a lot simpler approach, using the graphics card, that does not scale well for big scenes as raytracing does, but that worked just fine for these small scenes in this project.

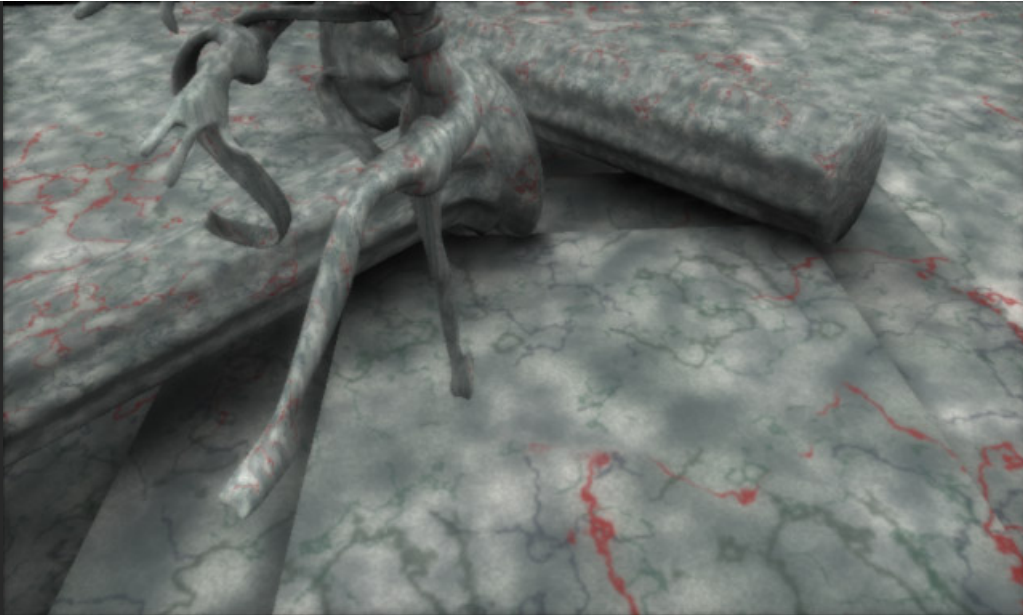


The ambient occlusion contribution of the "Stone" scene.

Once you have your Ambient Occlusion into the scene is time to enjoy with the shaders. Let's make a comparison to test the impact of the new lighting technique. First image below shows basic shading of a simple texture modulated by the diffuse term of a single directional light according to classical OpenGL shading. We could add some shadow mapping to this to gain a bit of depth perception since the real 3d position of the objects is quite ambiguous otherwise. Then, below, the same image with the same texture modulated not by the diffuse term but by the Ambient Occlusion calculated in the image shown before.



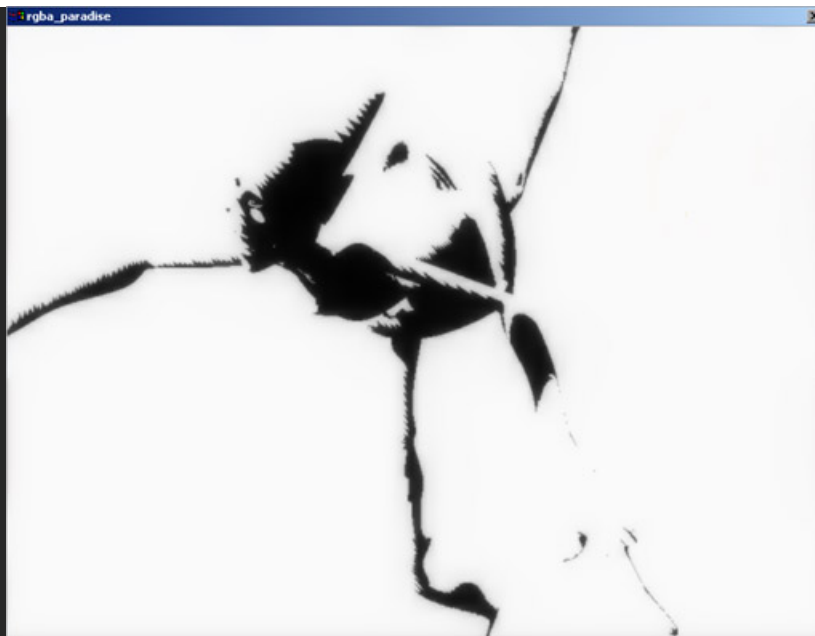
Traditional OpenGL shading with one directional light source.



Replacement of the diffuse term with the Ambient Occlusion.

Note the difference in the realism of the image, and the "soft shadow" effect created by the occlusion of the sky below the columns. This is nice, but we can use this new information we have about light occlusion in other ways for sure. In fact, Ambient Occlusion was originally designed for replacement of the ambient term in the traditional direct lighting model (OpenGL model). That term is normally a constant color, but can be replaced with the Ambient Occlusion and then continue doing regular OpenGL lighting. Let's have a look to the process in another scene of the demo.

For the "Wax" scene of the demo the first part to be done was the main statue and the shader. This first image shows the shadow map for the direct diffuse lighting coming from a directional light source.



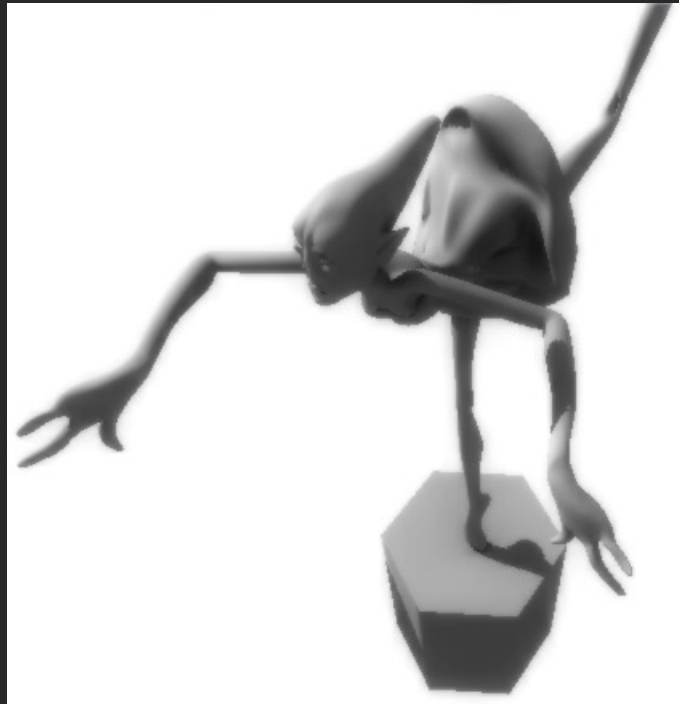
Just shadow rendering.

Next image shows the normal OpenGL shading applied to the model, making use of the shadow map to cancel the diffuse illumination. Note how the shadow map artifacts (due to small resolution shadow map) mostly disappeared (thankfully).



Rutinal OpenGL rendering, making use of the shadow map information.

And finally the Ambient Occlusion is used to replace the constant ambient term to fake the indirect lighting of the scene. Note once more the increase on the realism of the image.



Realtime use of the Ambient Occlusion to simulate indirect lighting.

inigo quilez - learning computer graphics since 1994